

Google OAuth2 — Implémentation dans SUPMEAL

Evan Morvant

Ce document explique comment l'authentification Google est implémentée et comment la configurer en local.

Aperçu

- Libs : `passport` , `passport-google-oauth20` , `jsonwebtoken` .
- Endpoints backend :
 - GET `/api/v1/auth/oauth/google` — démarre le flux : signe un `state` et redirige vers Google.
 - GET `/api/v1/auth/oauth/google/callback` — reçoit le `code` , l'échange contre le profil Google, crée ou retrouve l'utilisateur, puis **redirige vers le client** avec les jetons.
 - POST `/api/v1/users/me/oauth/google` — pour un utilisateur **déjà connecté** : renvoie l'URL d'autorisation permettant de rattacher son compte Google à son profil.
- Frontend : le bouton « S'inscrire avec Google » est un simple lien vers l'endpoint de départ. Après consentement, le navigateur revient sur `/oauth/callback#accessToken=...&refreshToken=...` ; la page lit le **fragment**, stocke les jetons, nettoie l'URL et redirige vers l'accueil.

L'échange du code se fait entièrement côté serveur. Le `client_secret` ne quitte jamais l'API : le client web ne voit passer que les jetons SUPMEAL, jamais ceux de Google.

Prérequis

- Un compte Google et l'accès à la [console Google Cloud](#).
- Documentation officielle : <https://developers.google.com/identity/protocols/oauth2/web-server>

Éléments à créer :

- un projet Google Cloud ;
- un écran de consentement OAuth ;
- un identifiant client OAuth 2.0 de type **Application Web**.

L'URI de redirection à déclarer est celle de **l'API**, pas celle du client :

`http://localhost:4000/api/v1/auth/oauth/google/callback`

Mise en place pas-à-pas

1. Créer le projet

- Console Google Cloud → sélecteur de projet → *Nouveau projet* → le nommer, par exemple SUPMEAL .

2. Configurer l'écran de consentement

- *APIs & Services* → *OAuth consent screen*.
- Type **External** pour un test ; renseigner le nom de l'application et une adresse de contact.
- Tant que l'application est en mode *Testing*, ajouter les comptes Google autorisés dans *Test users* — sans quoi la connexion est refusée avec `access_denied` .

3. Créer l'identifiant client

- *APIs & Services* → *Credentials* → *Create Credentials* → *OAuth client ID*.
- Type d'application : **Web application**.
- **Authorized redirect URIs** : `http://localhost:4000/api/v1/auth/oauth/google/callback`
- Noter le **Client ID** et le **Client Secret** affichés à la création.

La prise en compte d'une nouvelle URI peut demander quelques minutes.

4. Renseigner les variables d'environnement

Dans le `.env` à la racine du dépôt :

```
GOOGLE_CLIENT_ID=<client_id>
GOOGLE_CLIENT_SECRET=<client_secret>
# Doit correspondre au préfixe de l'URI de redirection déclarée ci-dessus
API_PUBLIC_URL=http://localhost:4000
# Où l'API renvoie le navigateur une fois le flux terminé
CLIENT_ORIGIN=http://localhost:8080
```

Deux ports interviennent, et c'est normal : en Docker, le bouton passe par

Nginx sur `8080` , qui proxifie l'API, mais le **callback est déclaré sur**

4000 — c'est l'API qui le construit, à partir d' `API_PUBLIC_URL` . Le fournisseur revient donc directement sur l'API, qui renvoie ensuite le navigateur vers l'application.

`API_PUBLIC_URL` et `CLIENT_ORIGIN` ne sont pas décoratifs : l'URI de callback envoyée à Google est construite à partir du premier, et la redirection finale à partir du second. Une valeur qui ne correspond pas à ce qui est déclaré dans la console donne un `redirect_uri_mismatch` .

5. Relancer les services

```
docker compose up -d api
```

Aucune reconstruction d'image n'est nécessaire : seules les variables changent, et Compose recrée le conteneur pour les injecter.

Les stratégies Passport ne sont enregistrées **qu'au démarrage**, et seulement si les deux variables sont renseignées. Un redémarrage est donc indispensable après modification du `.env`.

6. Tester

- Ouvrir <http://localhost:8080/register> → « S'inscrire avec Google » → écran de consentement → retour automatique sur l'application, connecté.
- Depuis *Paramètres* → *Compte*, le bouton **Lier** rattache un compte Google à un profil existant. Le retour se fait sur `/oauth/callback#linked=google`, sans nouveaux jetons : la session en cours n'est pas remplacée.

Ce qui se passe en coulisses

Le paramètre `state` est un **JWT signé** valable 10 minutes, portant le fournisseur visé et, pour une liaison, l'identifiant de l'utilisateur connecté. Il remplit deux rôles :

- protection contre le CSRF — un callback dont le `state` n'est pas signé par l'API est rejeté (`INVALID_STATE`) ;
- transport de l'identité pour la liaison de compte — une navigation vers Google ne peut pas porter d'en-tête `Authorization`, l'information doit donc voyager par le `state`.

À la fin du flux, l'utilisateur est résolu dans cet ordre :

- un compte OAuth déjà rattaché à ce `providerUserId` → on reprend l'utilisateur associé ;
- sinon, un compte SUPMEAL portant la même adresse e-mail → **on rattache** le compte Google à ce profil ;
- sinon, création d'un utilisateur sans mot de passe.

Un compte créé ainsi n'a pas de mot de passe local. Il peut s'en donner un depuis *Paramètres* → *Mot de passe*, en laissant vide le champ « mot de passe actuel ».

Si ça ne marche pas

Symptôme	Cause probable
503 PROVIDER_NOT_CONFIGURED	GOOGLE_CLIENT_ID OU GOOGLE_CLIENT_SECRET vide, ou API non redémarrée depuis la modification du <code>.env</code> .
redirect_uri_mismatch	L'URI déclarée dans la console diffère de <code>API_PUBLIC_URL + /api/v1/auth/oauth/google/callback</code> , au caractère près.

Symptôme	Cause probable
Retour sur <code>/oauth/callback#error=OAUTH_ACCOUNT_TAKEN</code>	Ce compte Google est déjà rattaché à un autre profil SUPMEAL. Le délier depuis ce profil avant de le relier.
Retour sur <code>/oauth/callback#error=state_invalide</code>	Plus de 10 minutes entre le départ et le retour, ou secrets JWT changés entre-temps.
<code>access_denied</code>	Compte non déclaré dans <i>Test users</i> alors que l'écran de consentement est en mode <i>Testing</i> .
Retour sur le client mais session absente	<code>CLIENT_ORIGIN</code> ne pointe pas sur l'application réellement ouverte (8080 en Docker, 5173 en développement).

Documentation réalisée par Evan Morvant.